

CSC 480: Artificial Intelligence

3.3- Tree Search - Adversarial Search

Rodrigo Canaan
Assistant Professor
Computer Science Department
Cal Poly, San Luis Obispo
rcanaan@calpoly.edu

Jan 30, 2026

Adversarial Search

Adversarial Search

Minimax and Alpha-Beta pruning

Multiplayer games



source: https://www.ssbwiki.com/File:8-Player_Smash_Original_8.jpg (fair use)

Multiplayer games

- Games with 2+ players are harder to analyze
- We need to keep track of the current player
- Reward/goal is no longer defined by a single value
 - One solution is to instead define the reward as a tuple e.g. $[-1, 1]$ or $[1, 2]$ corresponding to the “utility” of each player

Multiplayer games - more players

- Games with 3+ players are **even harder**
- Modeling rewards as tuples still work but:
 - Actions that don't affect a player may affect others
 - Social factors alliances, king making, diplomacy etc.

Zero-Sum Games

- In zero-sum games, the utility of both players always adds to zero
 - Simplest case: single winner game (1 for win, -1 for loss, 0 for draw)
- Allows us to model the result as a single number.
 - Utility is read from the perspective of “player 1”
 - A **1** result (1st player wins) corresponds to **[1,-1]**
 - A **-1** result (2nd player wins) corresponds to **[-1,1]**
 - A **0** result (draw) corresponds to **[0,0]**

Constant-Sum Games

- A slightly more general type of game is **constant sum**
- Here all utilities add up to a constant
 - Analogy: players are competing for a share of a fixed “pie”
- Can always be reduced to zero-sum by adding a “fee” to enter
 - For example, if all utilities add up to 10, a fee of 5 to enter turns the game into zero-sum
 - This means we can assume the game is zero-sum rather than constant sum.

Minimax intuition

- With general utilities, each player tries to maximize their own reward
 - In particular, this creates situations where one player is indifferent among outcomes that another player cares about.
 - It also creates situations where one player playing sub optimally results in worse outcomes for a different player
 - Hard to model!
- With zero-sum, we can model the outcome of the game as a single value $\mathbf{u}$
 - One player (max) tries to maximize $\mathbf{u}$, the other (min) tries to minimize $\mathbf{u}$
 - This removes both problems above. Easier to model
 - Easier to model

Minimax Value

- ❖ Definition: minimax value of a state s with successors s_i

$$\text{Minimax}(s) = \begin{cases} U(s) & \text{if } s \text{ is terminal} \\ \max_{s_i \in \text{succ}(s)} \text{Minimax}(s_i, d-1) & \text{if MAX moves} \\ \min_{s_i \in \text{succ}(s)} \text{Minimax}(s_i, d-1) & \text{if MIN moves} \end{cases}$$

- ❖ It can be shown that if the two players play optimally from a state s , the final utility of the game will be $\text{Minimax}(s)$
- ❖ Strategy outline: compute Minimax recursively, choose node with best Minimax value
- ❖ For two-player zero-sum games, it can be shown this corresponds to optimal play (Nash equilibrium)

Minimax Pseudocode

function MINIMAX-DECISION(*state*) *returns an action*

$v \leftarrow \text{MAX-VALUE}(\textit{state})$

return the action in SUCCESSORS(*state*) *with value* v

function MAX-VALUE(*state*) *returns a utility value*

if TERMINAL-TEST(*state*) *then return* UTILITY(*state*)

$v \leftarrow -\infty$

for a, s *in* SUCCESSORS(*state*) **do**

$v \leftarrow \text{MAX}(v, \text{MIN-VALUE}(s))$

return v

function MIN-VALUE(*state*) *returns a utility value*

if TERMINAL-TEST(*state*) *then return* UTILITY(*state*)

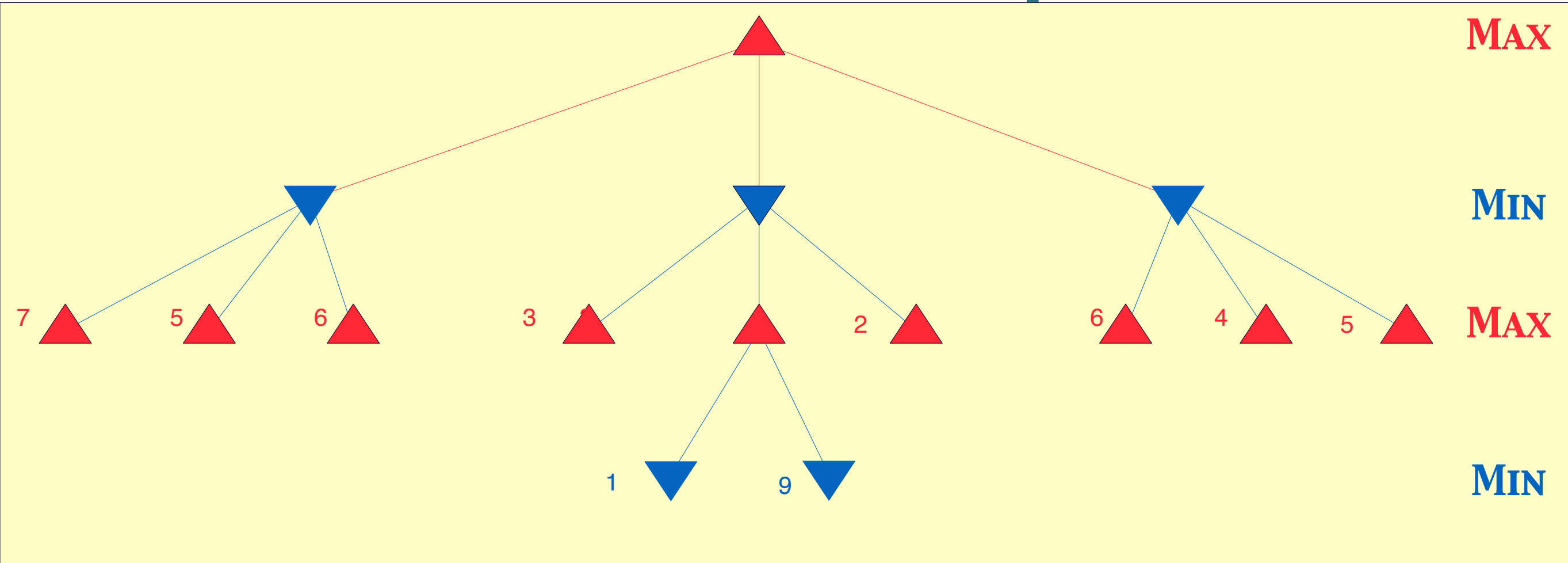
$v \leftarrow \infty$

for a, s *in* SUCCESSORS(*state*) **do**

$v \leftarrow \text{MIN}(v, \text{MAX-VALUE}(s))$

return v

Minimax Example



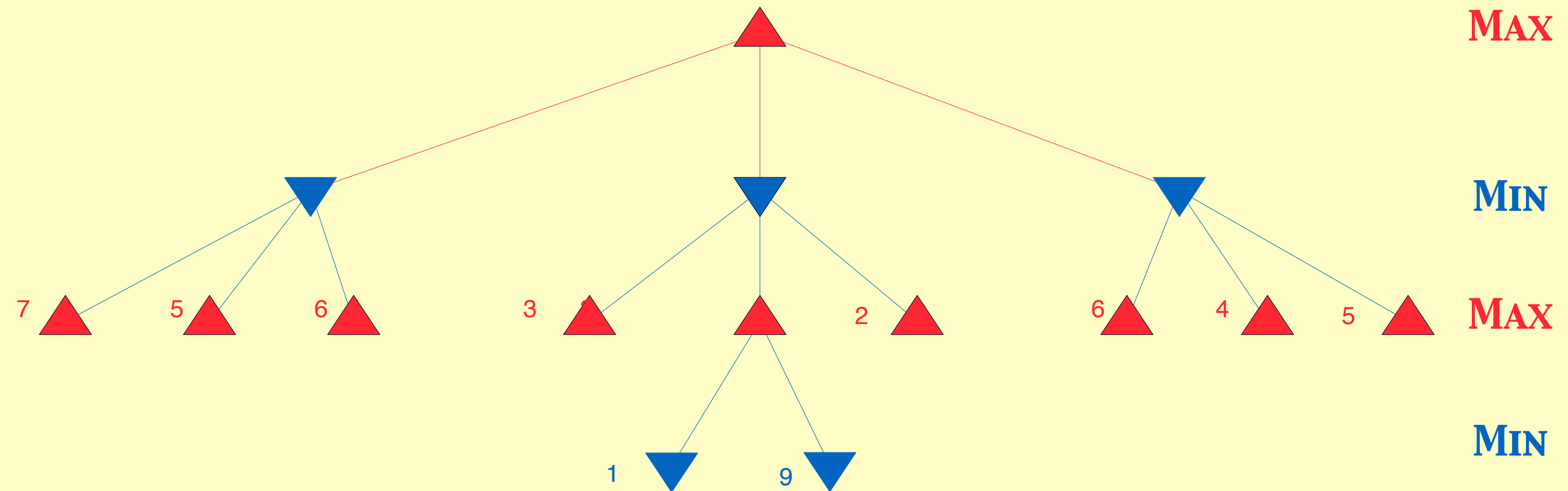
Legend:

▲ Max node

▼ Min node

1-9 Utility for the MAX player

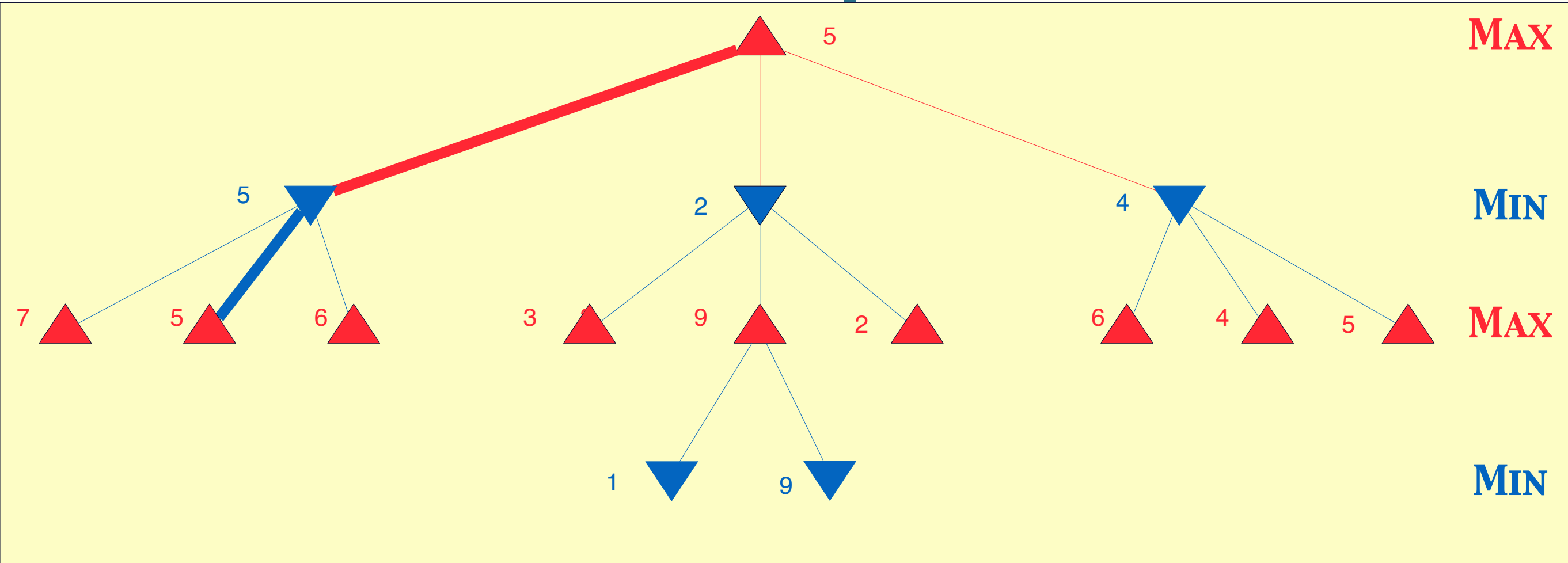
Minimax Example



Exercise (5 minutes):

- ❖ Fill in the minimax value of every non-terminal node
- ❖ Highlight the path resulting from both player's actions

Minimax Example Solution



Note: on a visual example, it's easy to go by “layers” from bottom to top, but it's important to remember Minimax explores the tree **Depth-First**

Minimax and Optimality

- ❖ **Minimax maximizes the worst-case outcome for MAX**
- ❖ **If MIN does not play optimally and the game is 2-player zero-sum game, MAX does even better**
 - ❖ But not necessarily as well as possible versus that specific sub-optimal player
 - ❖ See [Frank Lantz' explanation of the "Donkeyspace"](#) of trying to exploit sub-optimal strategies by playing sub-optimal yourself"
- ❖ **Minimax also does not distinguish between tied nodes**
 - ❖ But as long as both players are playing optimally, this doesn't matter (any tied node will lead to the minimax score)
 - ❖ Additional heuristics may need to be used if we assume one of the players is not optimal

MiniMax Properties

- ❖ **based on depth-first**
 - ❖ recursive implementation
- ❖ **time complexity is $O(b^m)$**
 - ❖ exponential in depth of the tree
- ❖ **space complexity is $O(b*m)$**
 - ❖ inherited from depth-first
 - ❖ If only one successor is generated at a time, can be $O(m)$ instead

b branching factor (possible moves - can be high for some games)

m maximum depth of the search tree (can be infinite, but often rules prohibit this)

Irrelevant Nodes in MiniMax

- ❖ the values of some of the leaf nodes are irrelevant for decisions at the next level
 - ❖ this also holds for decisions at higher levels
- ❖ as a consequence, under certain circumstances, some parts of the tree can be disregarded
 - ❖ it is possible to still make an optimal decision without considering those parts
 - ❖ these parts can be “pruned”

Pruning

- ❖ **discards parts of the search tree**
 - ❖ guaranteed not to contain good moves
 - ❖ guarantee that the solution is not in that branch or sub-tree
 - ❖ if both players make optimal (rational) decisions, they will never end up in that part of the search tree
 - ❖ sub-optimal moves by the opponent may lead into that part
- ❖ **results in substantial time and space savings**
 - ❖ as a consequence, longer sequences of moves can be explored
 - ❖ the leftover part of the task may still be exponential, however

Alpha-Beta Pruning

- ❖ **certain moves are not considered**
 - ❖ new states won't result in a better evaluation value than a move further up in the tree
 - ❖ they would lead to a less desirable outcome
- ❖ **applies to moves by both players**
 - ❖ α indicates the best choice for Max so far
 - ❖ never decreases
 - ❖ represents a lower bound
 - ❖ β indicates the best choice for Min so far
 - ❖ never increases
 - ❖ represents an upper bound
- ❖ **extension of the minimax approach**
 - ❖ results in the same sequence of moves as minimax, but with less overhead
 - ❖ prunes uninteresting parts of the search tree

Alpha-Beta Pseudocode

function ALPHA-BETA-SEARCH(*state*) *returns an action*

inputs: *state*, current state in game

$v \leftarrow \text{MAX-VALUE}(\text{state}, -\infty, +\infty)$

return the *action* in SUCCESSORS(*state*) with value *v*

function MAX-VALUE(*state*, α , β) *returns a utility value*

inputs: *state*, current state in game

α , the value of the best alternative for MAX along the path to *state*

β , the value of the best alternative for MIN along the path to *state*

if TERMINAL-TEST(*state*) **then return** UTILITY(*state*)

$v \leftarrow -\infty$

for *a, s* in SUCCESSORS(*state*) **do**

$v \leftarrow \text{MAX}(v, \text{MIN-VALUE}(s, \alpha, \beta))$

if $v \geq \beta$ **then return** *v*

$\alpha \leftarrow \text{MAX}(\alpha, v)$

return *v*

Alpha-Beta Pseudocode (cont.)

```
function MIN-VALUE(state,  $\alpha$ ,  $\beta$ ) returns a utility value  
  inputs: state, current state in game  
            $\alpha$ , the value of the best alternative for MAX along the path to state  
            $\beta$ , the value of the best alternative for MIN along the path to state  
  
  if TERMINAL-TEST(state) then return UTILITY(state)  
   $v \leftarrow +\infty$   
  for  $a, s$  in SUCCESSORS(state) do  
     $v \leftarrow \text{MIN}(v, \text{MAX-VALUE}(s, \alpha, \beta))$   
    if  $v \leq \alpha$  then return  $v$   
     $\beta \leftarrow \text{MIN}(\beta, v)$   
  return  $v$ 
```

Alpha-Beta Pruning

- ❖ **The roles of nodes with respect to α and β**
 - ❖ MAX nodes:
 - ❖ updates α
 - ❖ checks against β
 - ❖ MIN nodes
 - ❖ updates β
 - ❖ checks against α

Alpha-Beta Pseudocode (cont.)

function ALPHA-BETA-SEARCH(*state*) *returns an action*

inputs: *state*, current state in game

$v \leftarrow \text{MAX-VALUE}(\text{state}, -\infty, +\infty)$

return the *action* in SUCCESSORS(*state*) with value *v*

function MAX-VALUE(*state*, α , β) *returns a utility value*

inputs: *state*, current state in game

α , the value of the best alternative for MAX along the path to *state*

β , the value of the best alternative for MIN along the path to *state*

if TERMINAL-TEST(*state*) **then return** UTILITY(*state*)

$v \leftarrow -\infty$

for *a, s* in SUCCESSORS(*state*) **do**

$v \leftarrow \text{MAX}(v, \text{MIN-VALUE}(s, \alpha, \beta))$

if $v \geq \beta$ **then return** *v*

$\alpha \leftarrow \text{MAX}(\alpha, v)$

return *v*

function MIN-VALUE(*state*, α , β) *returns a utility value*

inputs: *state*, current state in game

α , the value of the best alternative for MAX along the path to *state*

β , the value of the best alternative for MIN along the path to *state*

if TERMINAL-TEST(*state*) **then return** UTILITY(*state*)

$v \leftarrow +\infty$

for *a, s* in SUCCESSORS(*state*) **do**

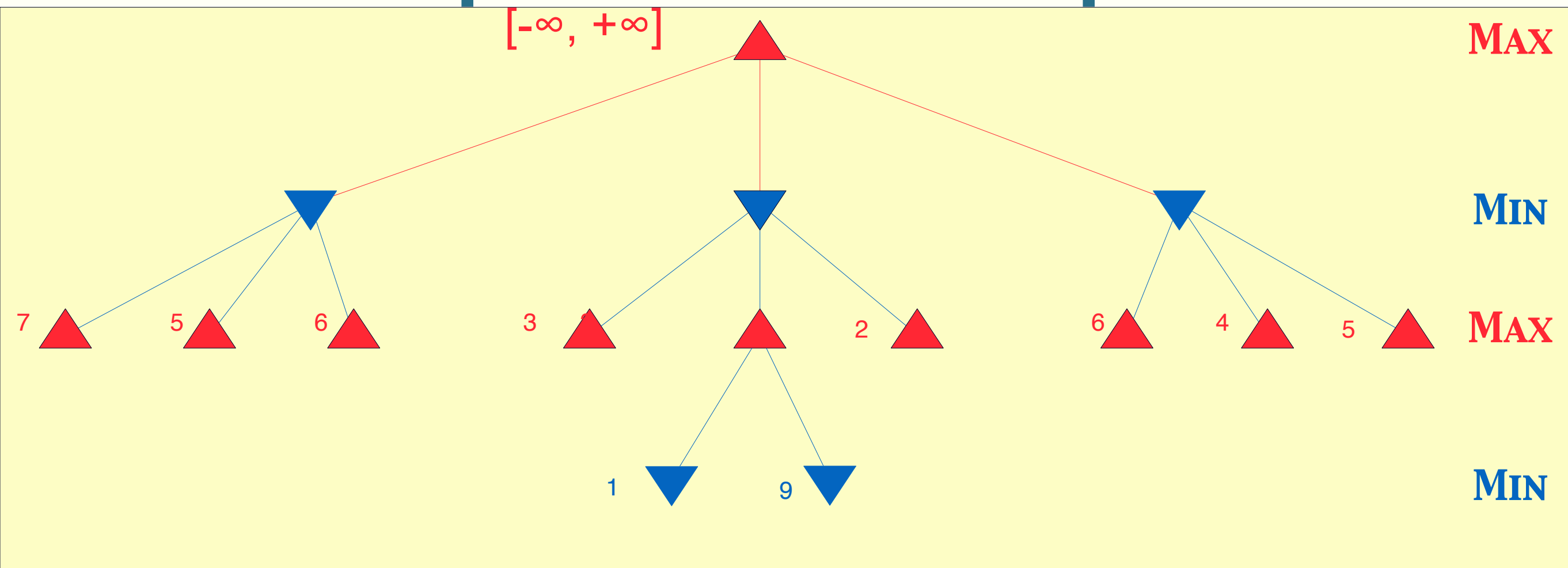
$v \leftarrow \text{MIN}(v, \text{MAX-VALUE}(s, \alpha, \beta))$

if $v \leq \alpha$ **then return** *v*

$\beta \leftarrow \text{MIN}(\beta, v)$

return *v*

Alpha-Beta Example



Legend:

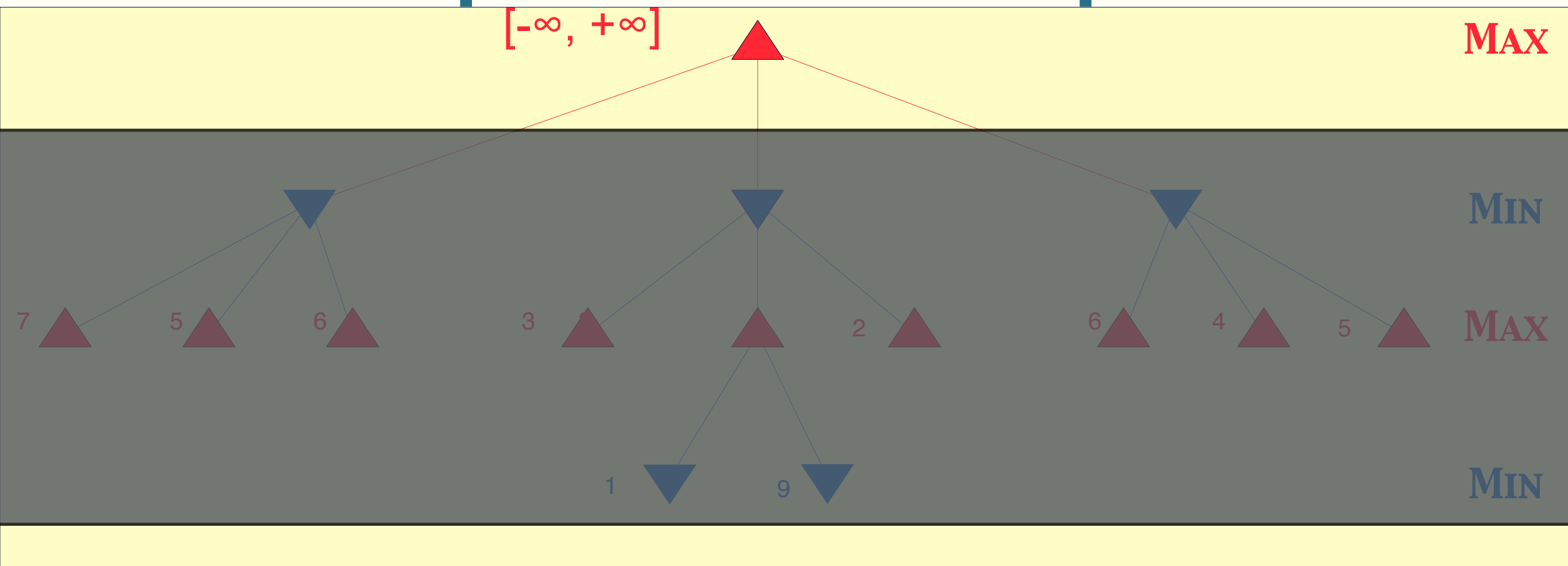
▲ Max node

▼ Min node

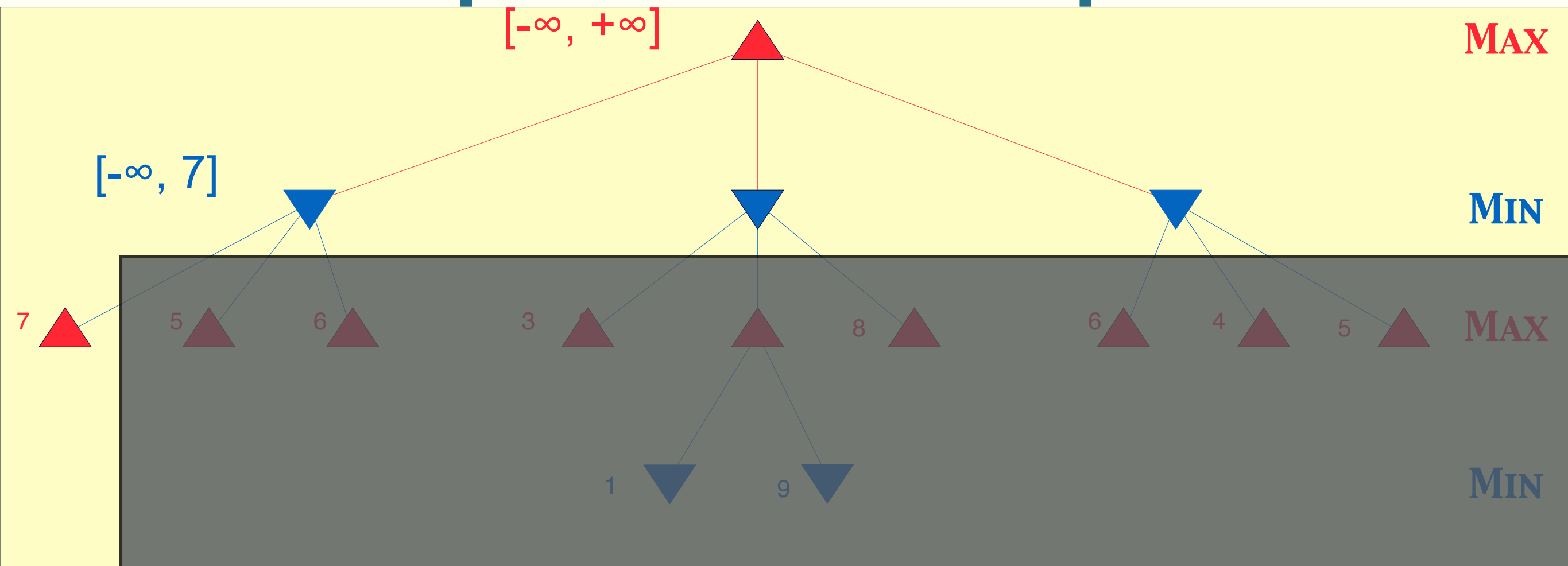
1-9 Utility

$[\alpha, \beta]$ Alpha and Beta values

Alpha-Beta Example

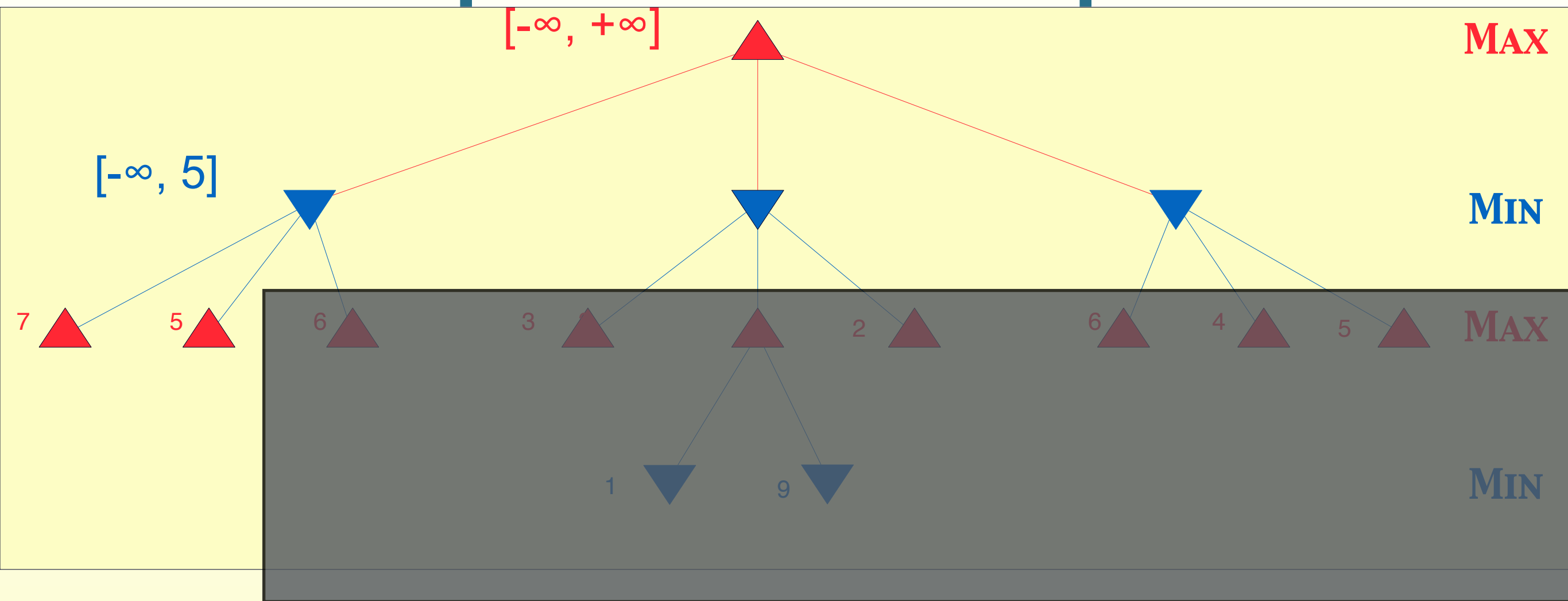


Alpha-Beta Example



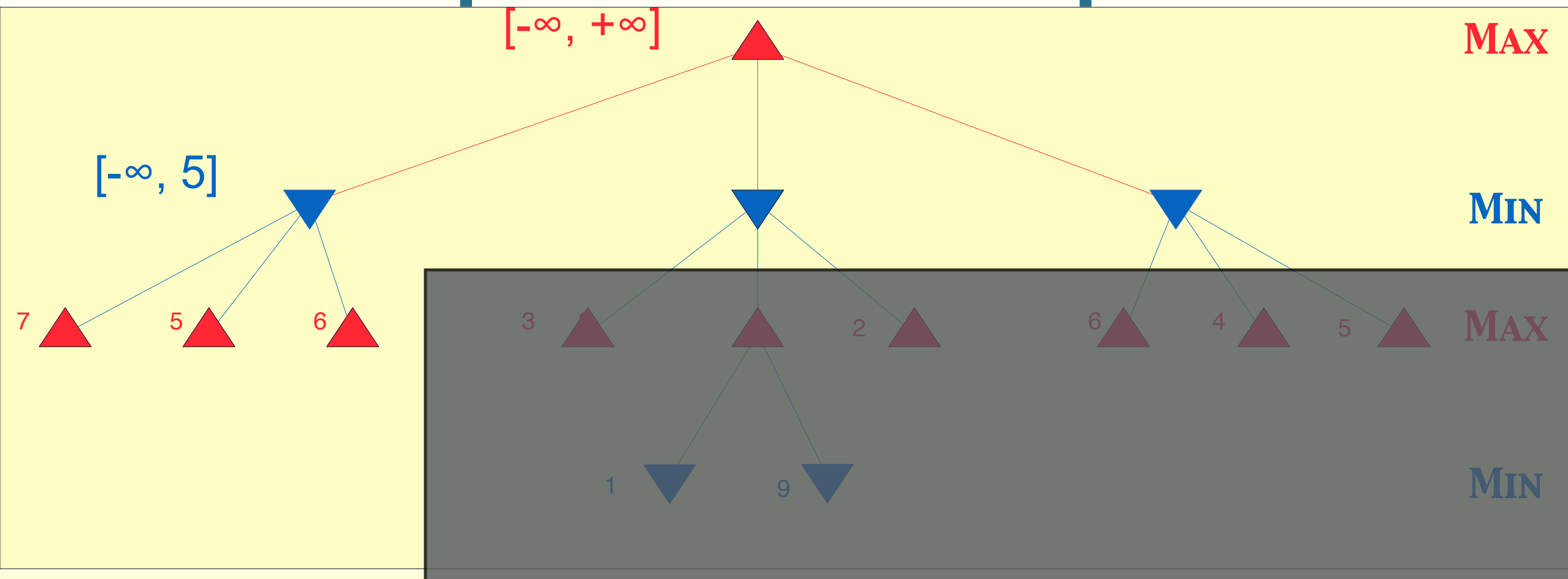
Regardless of the remaining children of the first MIN node, MIN will never choose < 7 at that node

Alpha-Beta Example



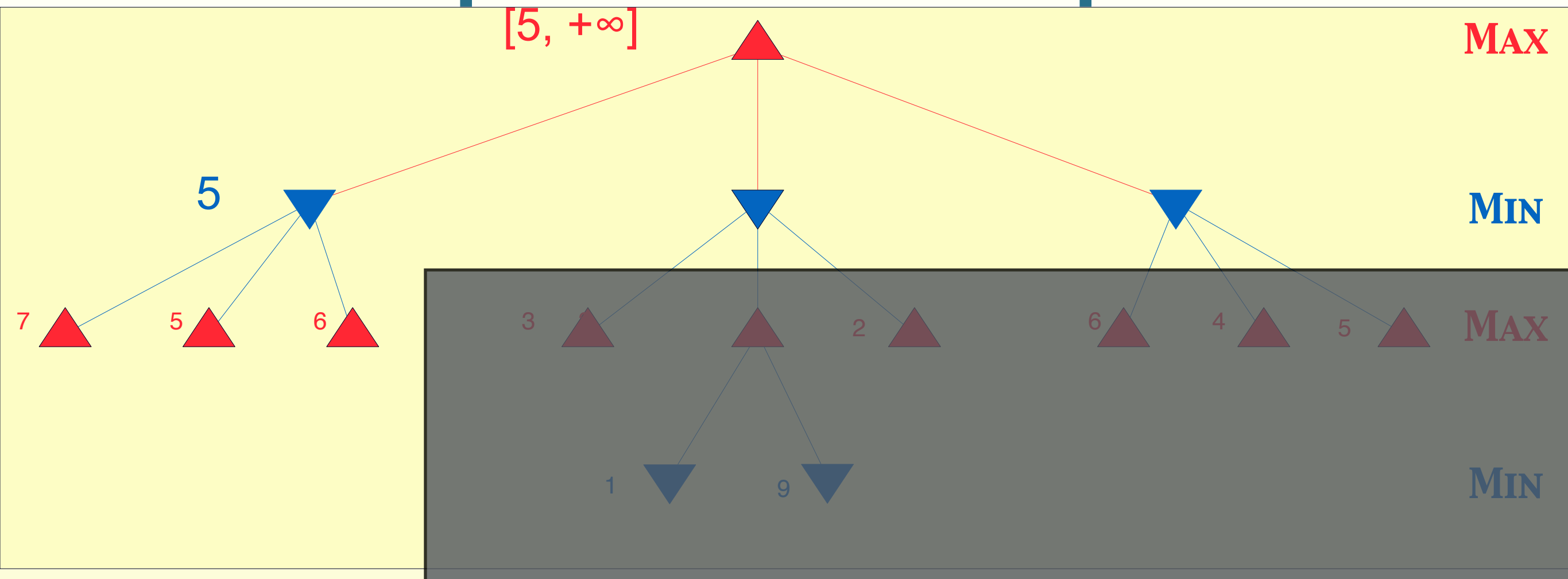
MIN node improves (reduces) its β value to 5

Alpha-Beta Example



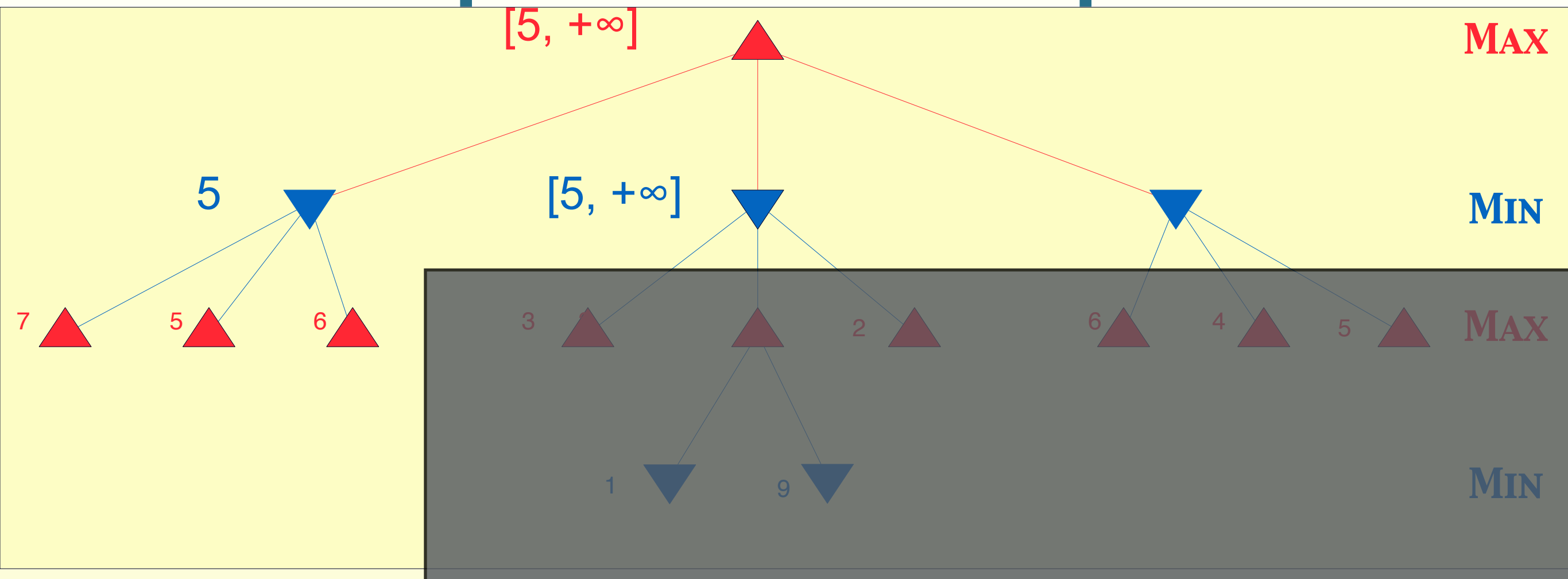
We've finished exploring the first MIN node, and β stays at 5

Alpha-Beta Example



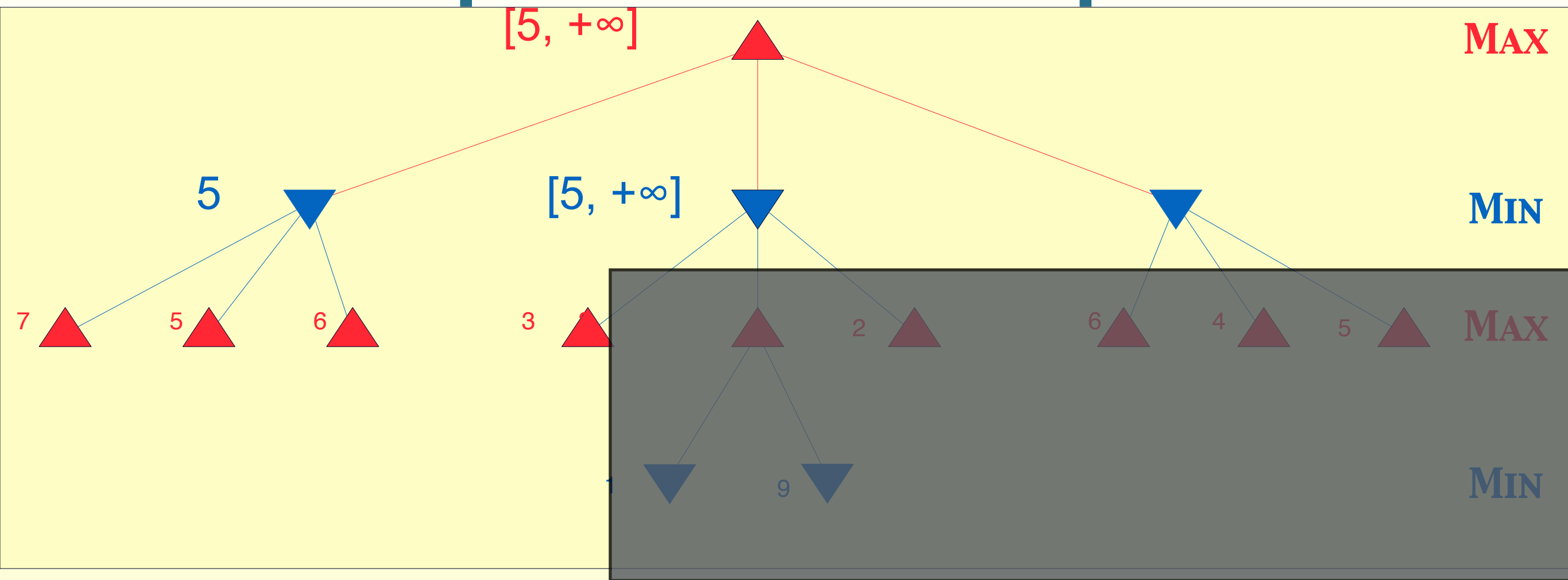
We update α at the starting node to 5: MAX will never pick a bid worse than 5

Alpha-Beta Example



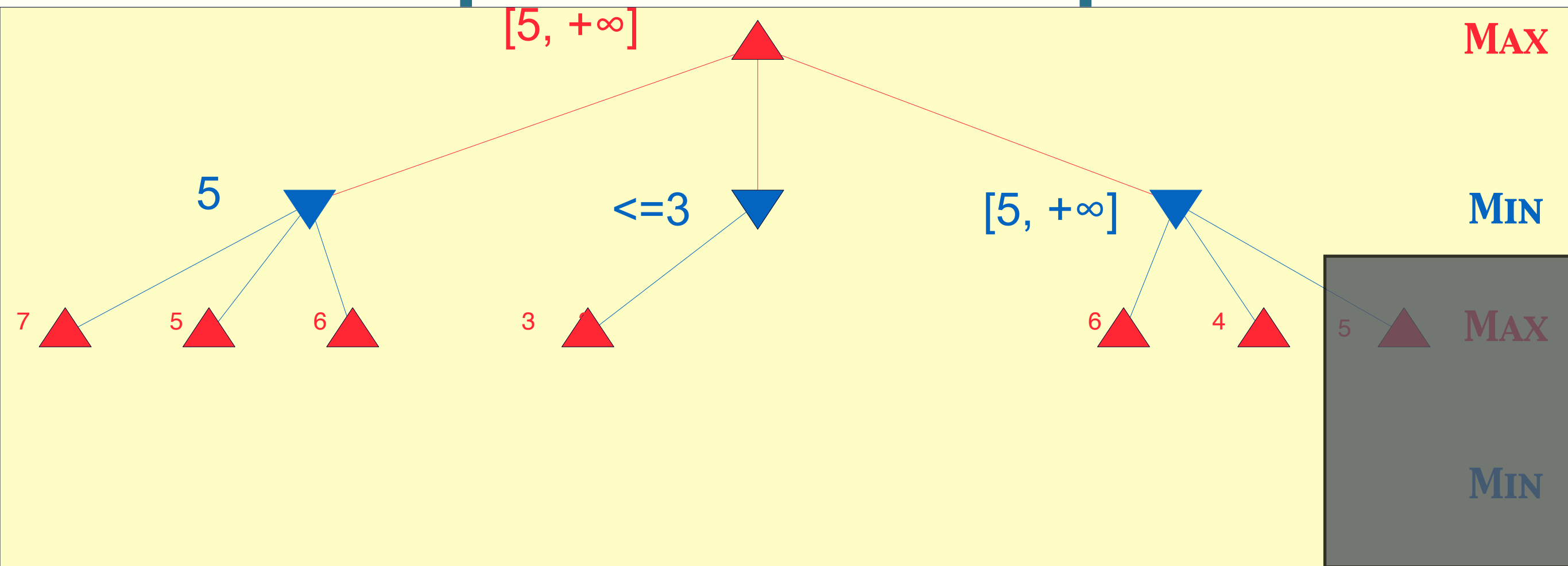
We propagate $\alpha=5$ to the next MIN node

Alpha-Beta Example



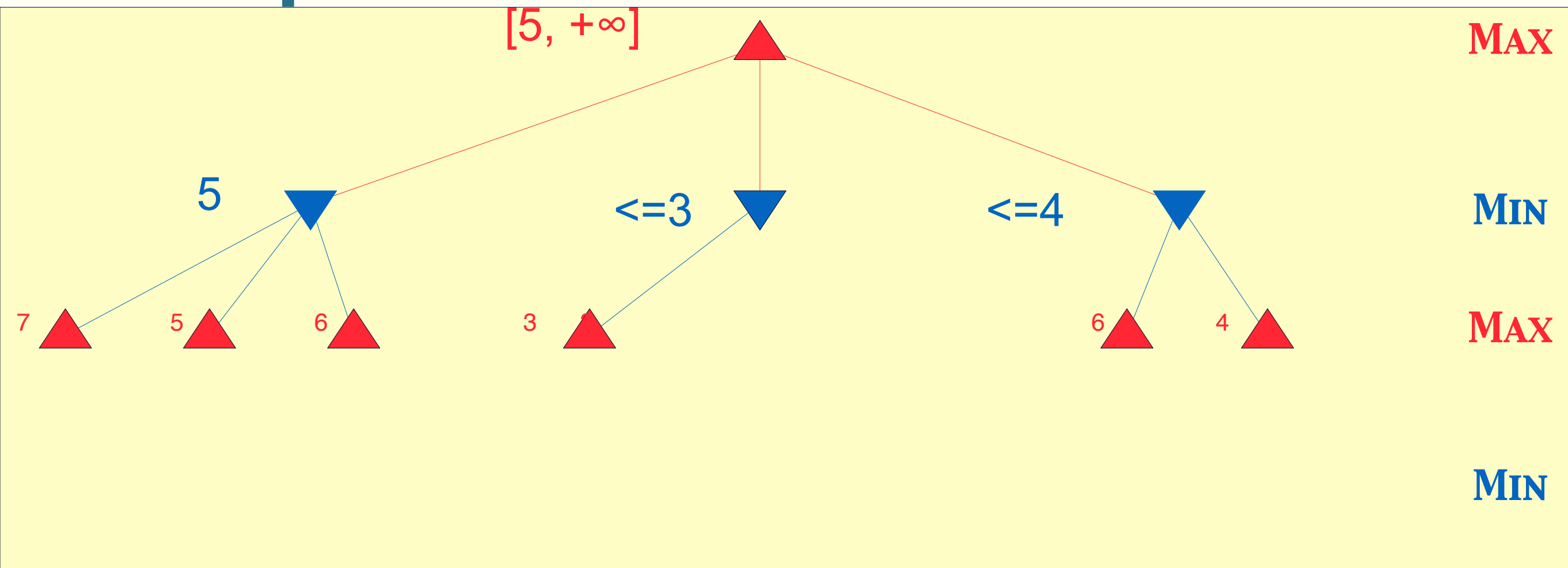
Min would like to update β to 3. But since this is $< \alpha$, we can stop here and prune the rest of the subtree

Alpha-Beta Example



Once again, we have a value for MIN that is $\leq \alpha$, so we can stop

Alpha-Beta Final Search Tree



Note that in the middle MIN node, we “got lucky” and only expanded one child (and pruned an entire subtree). In the rightmost MIN node, we got less lucky and only pruned one leaf node

Note on alpha-beta efficiency

- ❖ The efficiency of alpha-beta search depends on how many nodes are pruned in a practical scenario
- ❖ The order in which children of a node are visited matters
 - ❖ For MAX nodes, you ideally want children with _____ value visited first
 - ❖ For MIN nodes, you ideally want children with _____ value visited first

Note on alpha-beta efficiency

- ❖ The efficiency of alpha-beta search depends on how many nodes are pruned in a practical scenario
- ❖ The order in which children of a node are visited matters
 - ❖ For MAX nodes, you ideally want children with **greater** value visited first
 - ❖ For MIN nodes, you ideally want children with **smaller** value visited first
- ❖ Heuristics can help!

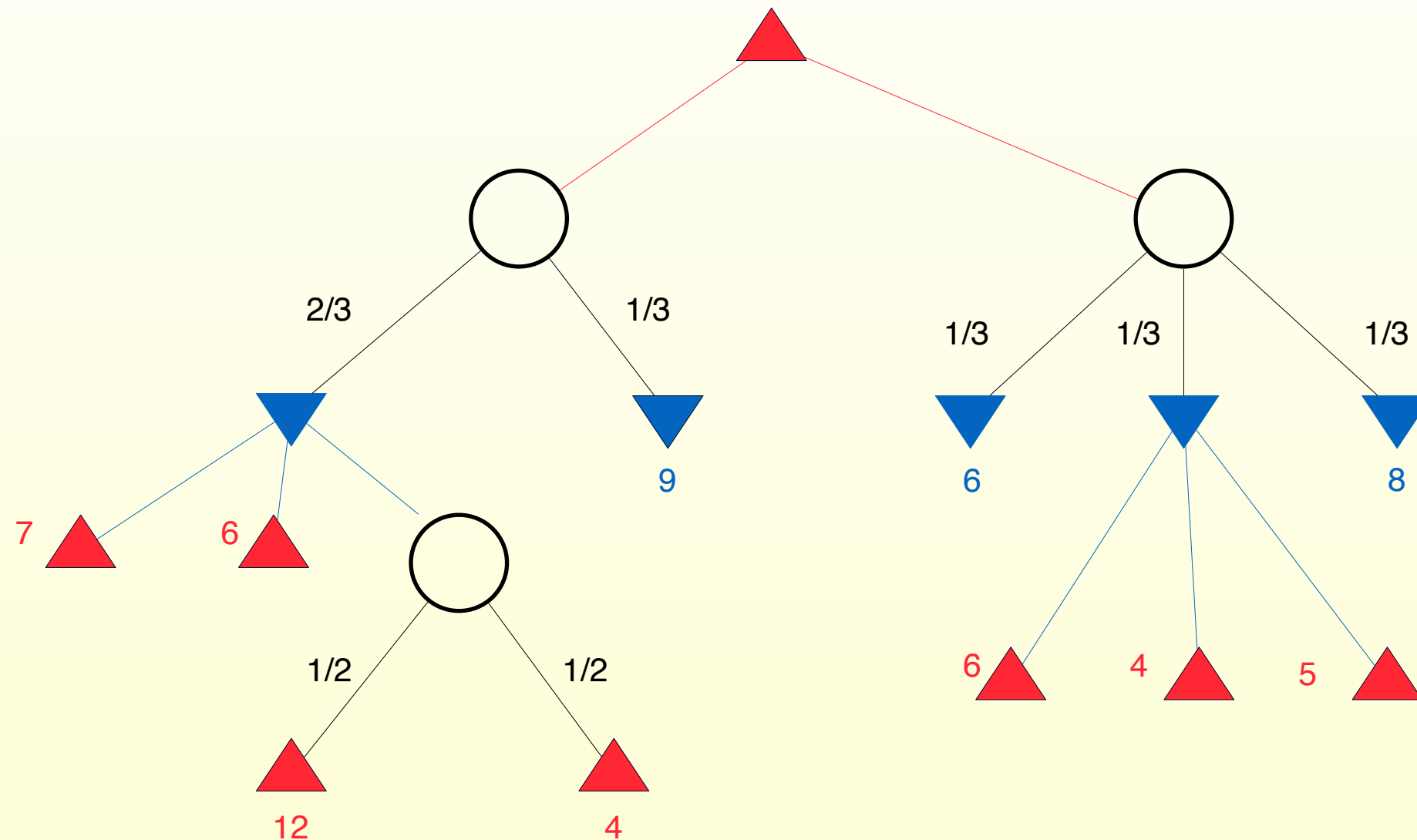
Expectiminimax

- Variation of Minimax to address stochastic environments
- In addition to min nodes, max nodes and terminal nodes, **chance nodes** are introduced
- A chance node return the expected value of its children (rather than the min or max)

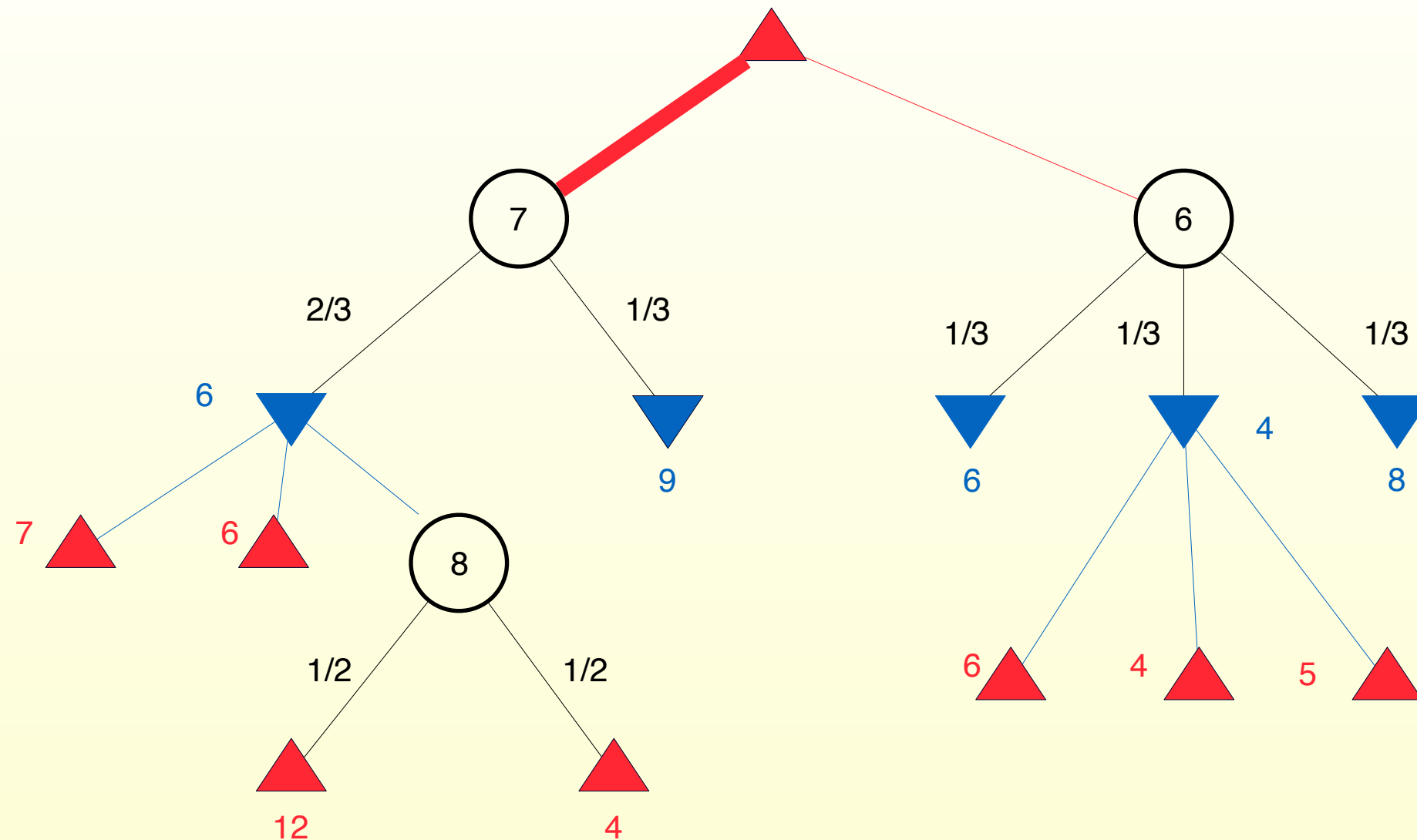
Expectiminimax

$$\text{Expectiminimax}(s) = \begin{cases} U(s) & \text{if } s \text{ is terminal} \\ \max_{s_i \in \text{succ}(s)} \text{Expectiminimax}(s_i) & \text{if } s \text{ is a MAX node} \\ \min_{s_i \in \text{succ}(s)} \text{Expectiminimax}(s_i) & \text{if } s \text{ is a MIN node} \\ \sum_{s_i \in \text{succ}(s)} P(s_i \mid s) \text{Expectiminimax}(s_i) & \text{if } s \text{ is a chance node} \end{cases}$$

Expectiminimax example



Expectiminimax example



Minimax with heuristic cutoff

- If the problem space is too big to fully explore, we can use an **evaluation heuristic** to cut the search early and return an approximate value

$$\text{Minimax}(s, d) = \begin{cases} U(s) & \text{if } s \text{ is terminal} \\ h(s) & \text{if } d = 0 \\ \max_{s_i \in \text{succ}(s)} \text{Minimax}(s_i, d - 1) & \text{if MAX moves} \\ \min_{s_i \in \text{succ}(s)} \text{Minimax}(s_i, d - 1) & \text{if MIN moves} \end{cases}$$

- Here, d is the (remaining) cutoff depth, and we call a node with $d=0$ a **cutoff node**

Minimax with heuristic cutoff

- Note: we interpret the heuristic differently than in A^*
 - in A^* : as an estimate of distance to the goal
 - In minimax: as an estimate of the utility of the eventual terminal state if both players play optimally
- In addition to early cutoffs, heuristics can also be used to establish move order for pruning

Minimax with heuristic cutoff

- With heuristic cutoff, we lose some guarantees:
 - The returned value (and corresponding root action) can't be trusted to be optimal
 - Alpha-beta pruning may lead to different values (and actions) being calculated
- However, with sophisticated heuristics we can get great results in some domains
 - Example: DeepBlue in Chess
- *Quiescence search* is a common addition to alleviate the horizon effect
 - We avoid terminating search at “swingy” states (e.g. captures in Chess)